

- Amazon ECS and AWS Fargate
- ▶ Getting Started
- ▶ Fundamentals
- ▶ Auto Scaling
- ▶ Networking
- ▶ Observability
- ▶ Security
- ▶ Automation
- ▼ **Storage**
 - Amazon EFS Volume with Fargate
 - Verify Amazon EFS Volume
 - Congratulations!

Storage

 **Important**
You must have completed the following chapters as pre-requisites for this lab:

- [Fundamentals](#)
- [ECS Service Connect](#)

Amazon ECS supports the following volume types for use with your ECS tasks:

- Amazon EBS
- Amazon EFS
- Amazon FSx for Windows File Server
- Docker Volumes
- Bind mounts

For detailed information about all available volume options, please refer to [this documentation](#). Let's examine the most commonly used volume types:

Volume	Supported launch type	Persistence	Description	Use cases
Amazon EBS	Fargate, Amazon EC2	Can be persisted when attached to a standalone task. Ephemeral when attached to a task maintained by a service	Cost-effective, durable, high-performance	Transactional workloads, log processing, ET L workloads
Amazon EFS	Fargate, Amazon EC2	Persistent	Simple, scalable, persistent	Data analytics, media processing, content management, web serving

If you've completed the pre-requisites chapter, you currently have an **Assets service** running with a single task. This configuration presents a critical single point of failure. Moreover, the current setup requires a complete application redeployment whenever product images are added or removed.

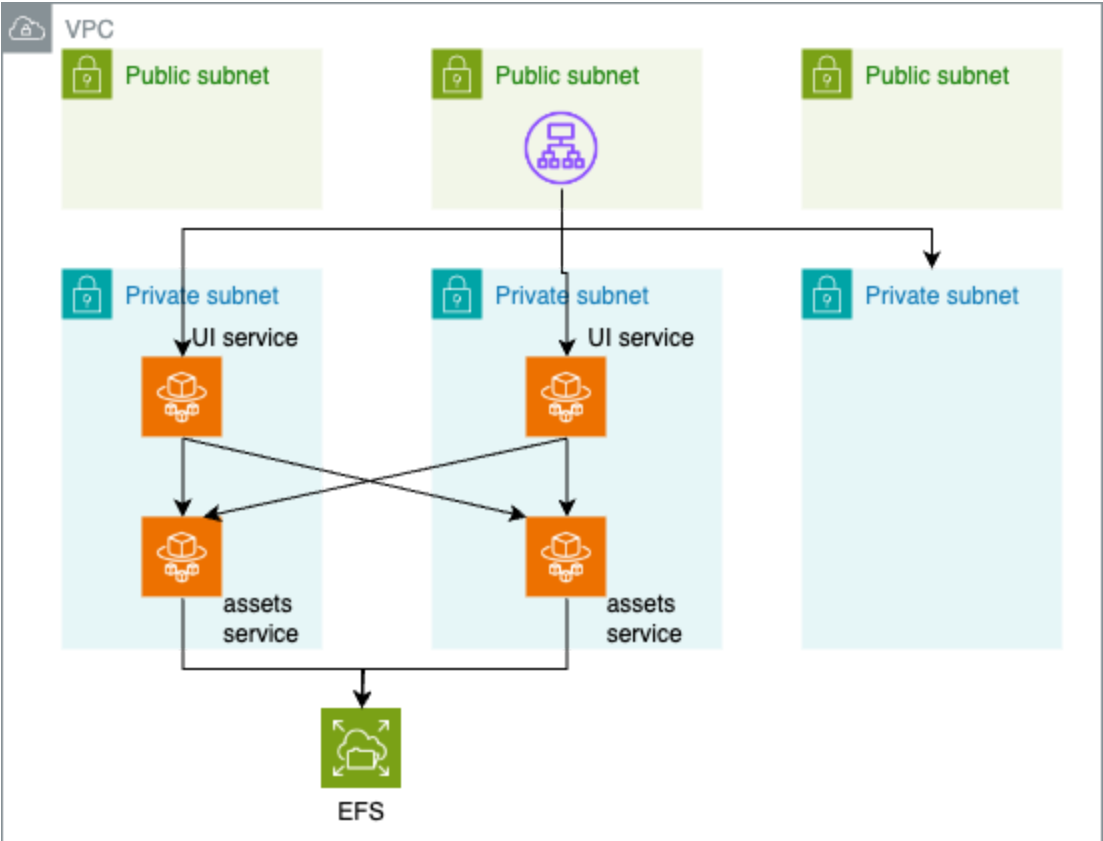
To enhance the reliability and scalability of the Assets service, we'll implement the following improvements:

- Deploy at least two Assets service tasks to ensure high availability
- Implement Amazon EFS as a shared storage solution for product images

Using Amazon EFS offers several key advantages in this scenario:

- Multiple scaled-out tasks can simultaneously access the same image files from EFS, facilitating seamless application scaling
- Centralized file storage eliminates the need for application redeployment when modifying image content

After implementing these changes, your architecture will look like this:



Previous

Next

Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose “Customize” or “Decline” to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose “Accept” or “Decline.” To make more detailed choices, choose “Customize.”

Accept

Decline

Customize

Amazon EFS Volume with Fargate

In this section, we will walk through the process of mounting an Amazon EFS volume to the **Assets service** and updating the product images.

To mount the EFS volume to the Assets application container, we need to add the following mount point configuration to the task definition:

```
"mountPoints": [{
  "sourceVolume": "efsVolume",
  "containerPath": "/usr/share/nginx/html/assets"
}]
```

Let's understand this configuration:

- sourceVolume: Specifies the name of the EFS volume to be mounted
- containerPath: Defines the mount point within the Assets container where the EFS volume will be attached. In this case, it's */usr/share/nginx/html/assets*, which is the directory for storing product images.

We also need to define the EFS volume configuration in the task definition:

```
"volumes": [{
  "name": "efsVolume",
  "efsVolumeConfiguration": {
    "fileSystemId": "$EFS_ID",
    "rootDirectory": "/",
    "transitEncryption": "ENABLED",
    "authorizationConfig": {
      "iam": "ENABLED"
    }
  }
}]
```

Since we'll need to verify the EFS volume mount through an interactive session later, we must configure the task role:

```
"taskRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskRole",
```

Let's update the ECS task definition for the Assets service:

```
1 cat << EOF > retail-store-ecs-asset-storage-taskdef.json
2 {
3   "family": "retail-store-ecs-assets",
4   "executionRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskExecutionRole",
5   "taskRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskRole",
6   "networkMode": "awsvpc",
7   "requiresCompatibilities": [
8     "FARGATE"
9   ],
10  "cpu": "1024",
11  "memory": "2048",
12  "runtimePlatform": {
13    "cpuArchitecture": "X86_64",
14    "operatingSystemFamily": "LINUX"
15  },
16  "containerDefinitions": [
17    {
18      "name": "application",
19      "image": "public.ecr.aws/aws-containers/retail-store-sample-assets:0.7.0",
20      "cpu": 0,
21      "portMappings": [
22        {
23          "name": "application",
24          "containerPort": 8080,
25          "hostPort": 8080,
26          "protocol": "tcp",
27          "appProtocol": "http"
28        }
29      ],
30      "essential": true,
31      "mountPoints": [
32        {
33          "sourceVolume": "efsVolume",
34          "containerPath": "/usr/share/nginx/html/assets"
35        }
36      ],
37      "healthCheck": {
38        "command": [
39          "CMD-SHELL",
40          "curl -f http://localhost:8080/health.html || exit 1"
41        ],
42        "interval": 10,
43        "timeout": 5,
44        "retries": 3,
45        "startPeriod": 60
46      },
47      "logConfiguration": {
48        "logDriver": "awslogs",
49        "options": {
50          "awslogs-group": "retail-store-ecs-tasks",
51          "awslogs-region": "$AWS_REGION",
52          "awslogs-stream-prefix": "assets-service"
53        }
54      }
55    }
56  ],
57  "volumes": [
58    {
59      "name": "efsVolume",
60      "efsVolumeConfiguration": {
61        "fileSystemId": "$EFS_ID",
62        "rootDirectory": "/",
63        "transitEncryption": "ENABLED",
64        "authorizationConfig": {
65          "iam": "ENABLED"
66        }
67      }
68    }
69  ]
70 }
71 EOF
72
73 aws ecs register-task-definition --cli-input-json file://retail-store-ecs-asset-storage-taskdef.json
```

Now, let's update the Assets service. The following command will update the service to maintain two tasks using the new task definition, eliminating single points of failure (~ 5 min):

```
aws ecs update-service \
--cluster retail-store-ecs-cluster \
--service assets \
--desired-count 2 \
--task-definition retail-store-ecs-assets \
--enable-execute-command \
--force-new-deployment

echo "Waiting for service to stabilize..."

aws ecs wait services-stable --cluster retail-store-ecs-cluster --services assets
```

With these steps completed, the EFS volume is now successfully mounted to the Assets service tasks running on the ECS Fargate cluster. We can proceed to verify

Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose "Customize" or "Decline" to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose "Accept" or "Decline." To make more detailed choices, choose "Customize."

Accept

Decline

Customize

Amazon ECS Immersion Day

Amazon ECS and AWS Fargate

- ▶

Getting Started
- ▶

Fundamentals
- ▶

Auto Scaling
- ▶

Networking
- ▶

Observability
- ▶

Security
- ▶

Automation

- ▼

Storage
- ▶

Amazon EFS Volume with Fargate
- ▶

Verify Amazon EFS Volume

Congratulations!

Verify Amazon EFS Volume

Now that we have integrated Amazon EFS with Amazon ECS, let's verify how we can utilize the mounted EFS volume. Since our application doesn't yet have the "add/delete product" feature, we'll import the updated images directly through our IDE.

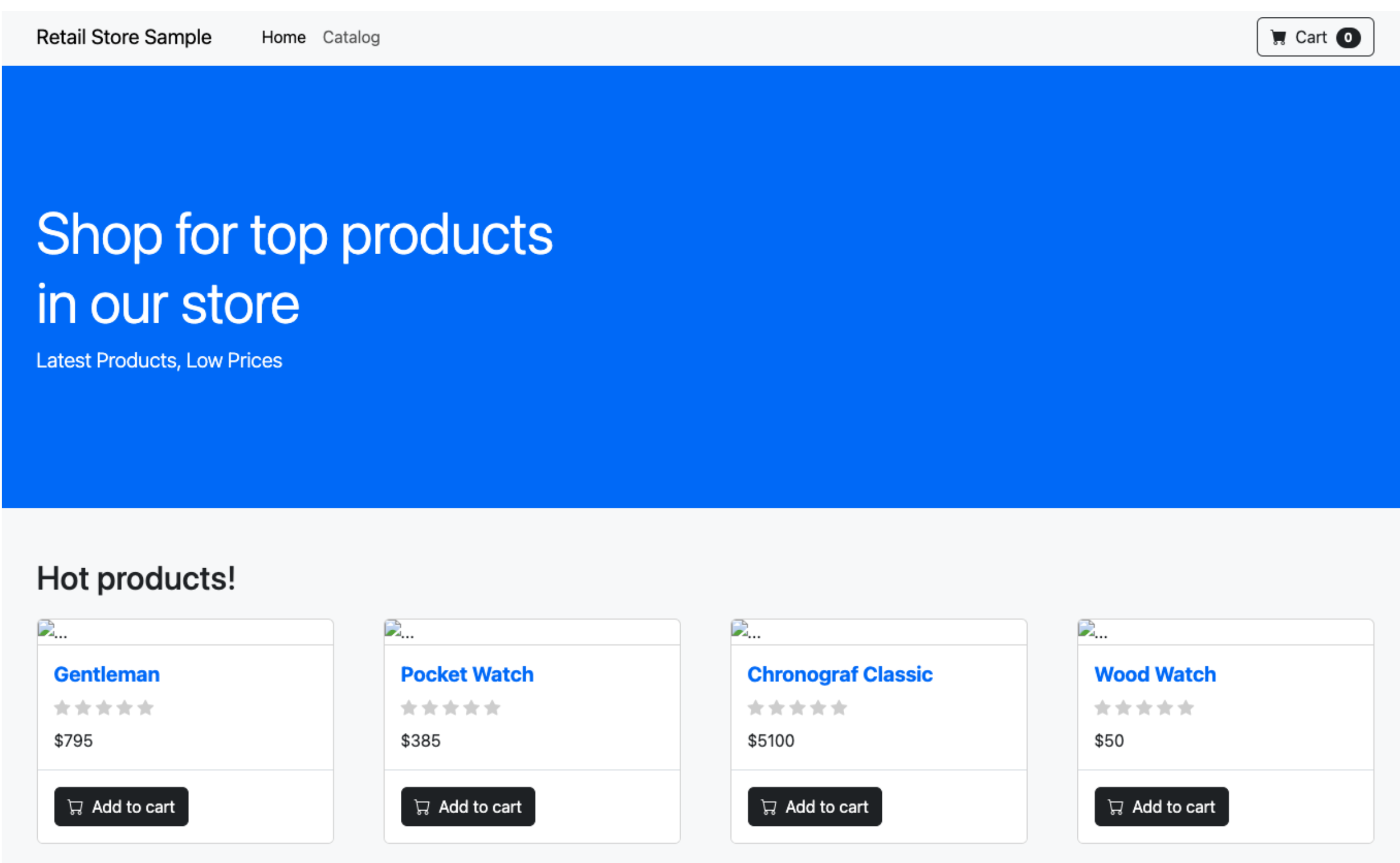
When you first access the application in a web browser, you'll notice broken image icons. This occurs because the EFS volume doesn't yet contain any images. You can verify this by accessing the application's URL:

```
export RETAIL_ALB=$(aws elbv2 describe-load-balancers --name retail-store-ecs-ui \
--query 'LoadBalancers[0].DNSName' --output text)

echo http://${RETAIL_ALB} ; echo
```

Tip

You may need to clear your browser's cache to see the changes.



Let's verify the directory contents within the container. First, start a `/bin/bash` interactive session in the running task:

```
ECS_EXEC_TASK_ARN=$(aws ecs list-tasks --cluster retail-store-ecs-cluster \
--service-name assets --query 'taskArns[]' --output text | \
xargs -n1 aws ecs describe-tasks --cluster retail-store-ecs-cluster --tasks | \
jq -r '.tasks[] | select(.enableExecuteCommand == true) | .taskArn' | \
head -n 1)

if [ -z ${ECS_EXEC_TASK_ARN} ]; then echo "ECS_EXEC_TASK_ARN is not correctly configured!"; else
aws ecs execute-command --cluster retail-store-ecs-cluster \
--task $ECS_EXEC_TASK_ARN \
--container application \
--interactive \
--command "/bin/bash"
fi
```

Then check the directory contents:

```
ls -al /usr/share/nginx/html/assets
```

You'll see that the directory is empty:

```
bash-5.2# ls -al /usr/share/nginx/html/assets
total 8
drwxr-xr-x 2 root root 6144 Oct 22 05:18 .
drwxr-xr-x 1 nginx nginx 4096 Jan 9 2024 ..
```

Exit the session:

```
exit
```

Import New Images to the EFS Volume

Since the EFS volume is mounted in our IDE environment, we can easily import the new images through the IDE.

First, let's check the current EFS mount information in the IDE:

```
df -h
```

You'll see output similar to this:

Filesystem	Size	Used	Avail	Use%	Mounted on
devtmpfs	4.0M	0	4.0M	0%	/dev
tmpfs	1.9G	0	1.9G	0%	/dev/shm
tmpfs	766M	516K	766M	1%	/run
/dev/nvme0n1p1	32G	3.3G	29G	11%	/
tmpfs	1.9G	77M	1.8G	4%	/tmp
/dev/nvme0n1p128	10M	1.3M	8.7M	13%	/boot/efi
tmpfs	383M	0	383M	0%	/run/user/0
127.0.0.1:/	8.0E	0	8.0E	0%	/home/ec2-user/environment/labs/efs/mount-point

Note that the EFS volume is mounted at `/home/ec2-user/environment/labs/efs/mount-point`. Let's copy the new images from `/home/ec2-user/environment/labs/efs/images` to the EFS mount point:

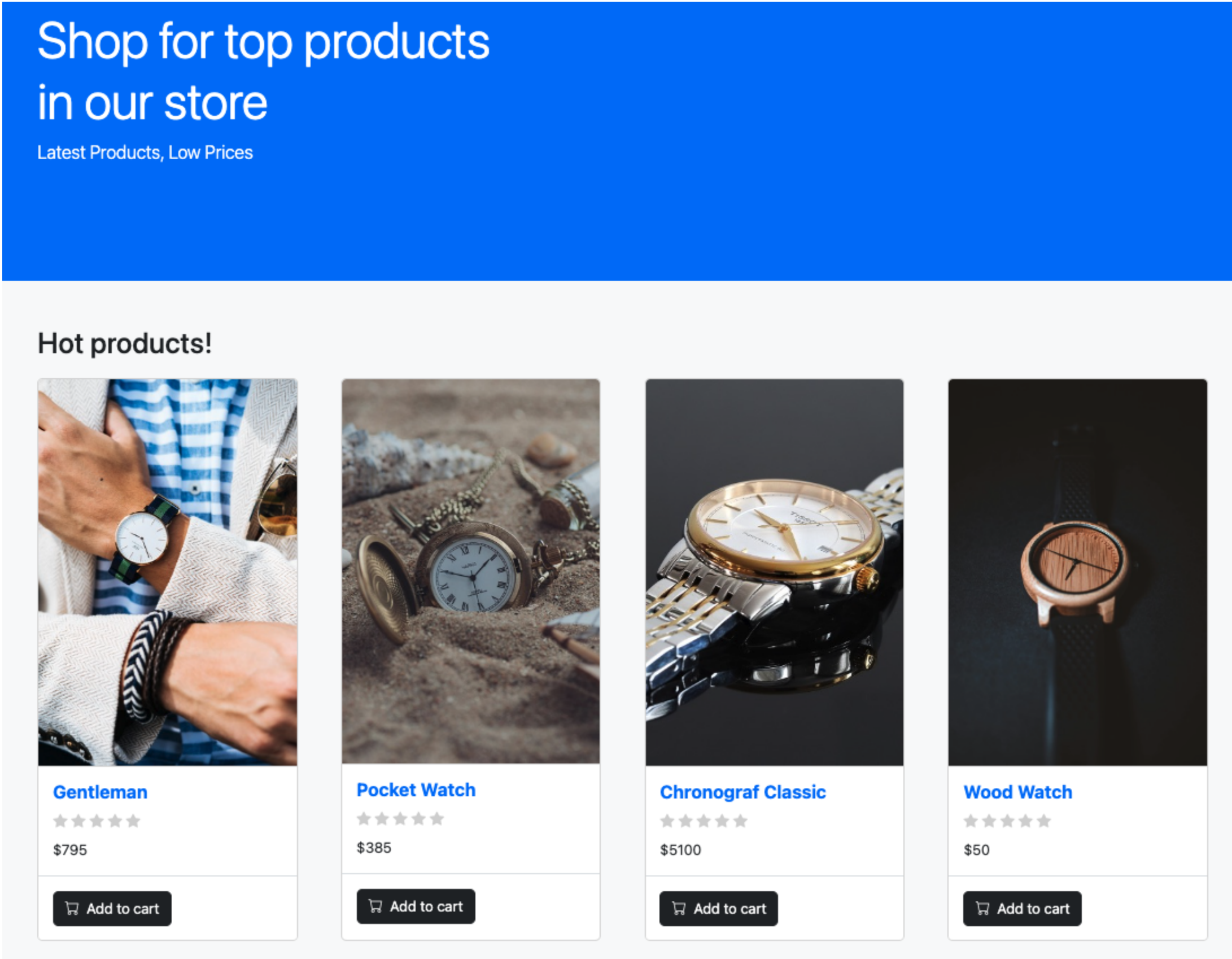
```
sudo cp ~/environment/labs/efs/images/* ~/environment/labs/efs/mount-point/
```

Verify the Updated Images

After copying the images, reload your browser to see the updated product images.

Tip

You may need to clear your browser's cache to see the changes.



Let's verify that the EFS volume is properly mounted in the container. Start another `/bin/bash` interactive session in the running task:

```
ECS_EXEC_TASK_ARN=$(aws ecs list-tasks --cluster retail-store-ecs-cluster \
--service-name assets --query 'taskArns[]' --output text | \
xargs -n1 aws ecs describe-tasks --cluster retail-store-ecs-cluster --tasks | \
jq -r '.tasks[] | select(.enableExecuteCommand == true) | .taskArn' | \
head -n 1)

if [ -z ${ECS_EXEC_TASK_ARN} ]; then echo "ECS_EXEC_TASK_ARN is not correctly configured!"; else
aws ecs execute-command --cluster retail-store-ecs-cluster \
--task $ECS_EXEC_TASK_ARN \
--container application \
--interactive \
--command "/bin/bash"
fi
```

Check the contents of the mounted EFS volume in the container:

```
ls -al /usr/share/nginx/html/assets
```

You should see the following output:

```
bash-5.2# ls -al /usr/share/nginx/html/assets
total 312
drwxr-xr-x 2 root root 6144 Oct 22 06:04 .
drwxr-xr-x 1 nginx nginx 4096 Jan 9 2024 ..
-rw-r--r-- 1 root root 43231 Oct 22 06:03 chrono_classic.jpg
-rw-r--r-- 1 root root 71709 Oct 22 06:03 gentleman.jpg
-rw-r--r-- 1 root root 53214 Oct 22 06:03 pocket_watch.jpg
-rw-r--r-- 1 root root 27844 Oct 22 06:03 smart_1.jpg
-rw-r--r-- 1 root root 45312 Oct 22 06:03 smart_2.jpg
-rw-r--r-- 1 root root 37255 Oct 22 06:03 smart_3.jpg
-rw-r--r-- 1 root root 20280 Oct 22 06:03 wood_watch.jpg
```

Exit the session:

```
exit
```

Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose “Customize” or “Decline” to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose “Accept” or “Decline.” To make more detailed choices, choose “Customize.”

Accept

Decline

Customize

